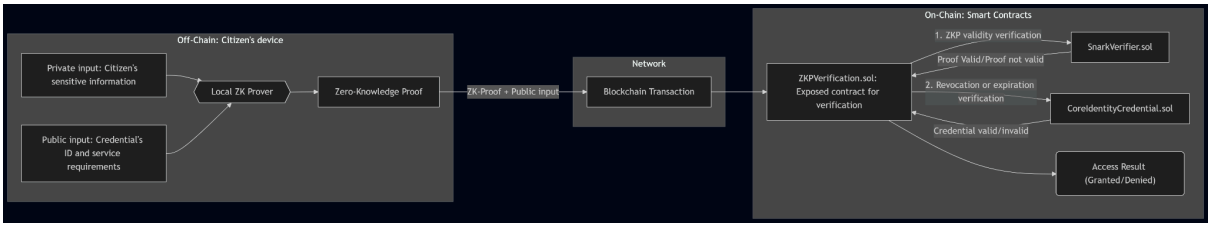
Palestine National Identity System - ZK Stack

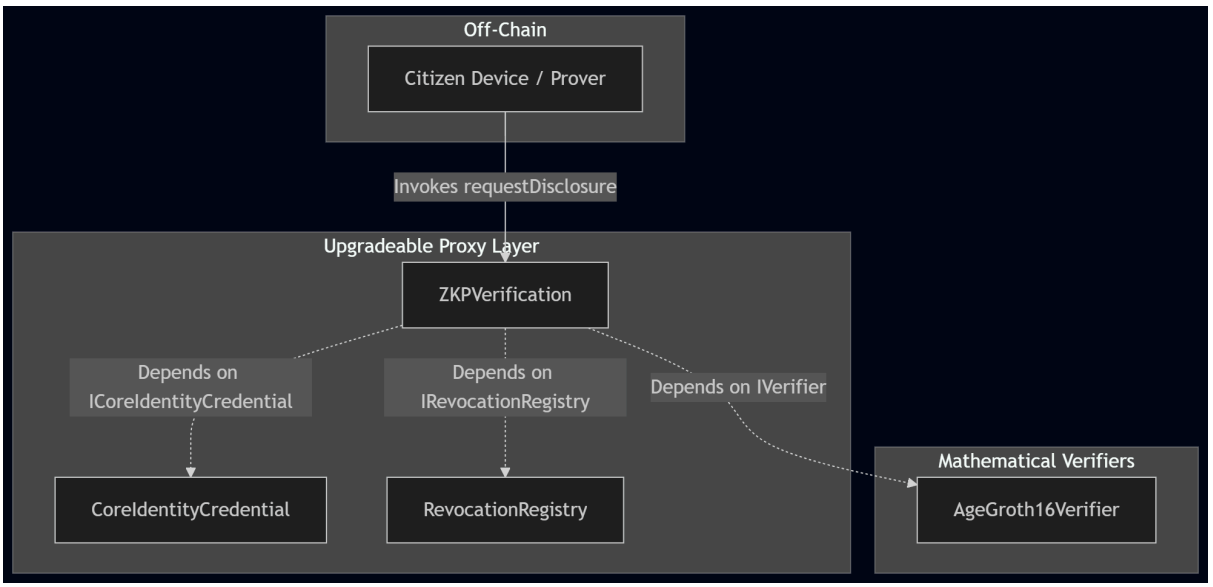
This report outlines the design, architecture, and implementation of the smart contract layer for the Palestine National Identity System. In the context of rebuilding national infrastructure, traditional centralized identity databases pose significant surveillance and security risks to the population. To address these critical vulnerabilities, this project introduces a sovereign, privacy-preserving digital identity solution leveraging Zero-Knowledge Proofs. By utilizing this cryptographic technology, citizens are empowered with plausible deniability, allowing them to securely prove specific claims to authorized service providers, such as being over a certain age, without ever exposing their underlying sensitive personal data.

The following sections will detail the foundational architectural choices made during the development of this ZK stack. We will explore the strict separation of concerns between the core identity anchoring layer and the privacy-preserving verification layer. Furthermore, the report discusses the implementation of selective disclosure mechanisms, role-based access controls for service providers, and the engineering behind a highly optimized, hybrid revocation registry. Finally, we will review the system's emergency security patterns, the design of the off-chain zero-knowledge circuits, and the comprehensive testing strategies employed to ensure the safety and efficiency of the entire infrastructure.

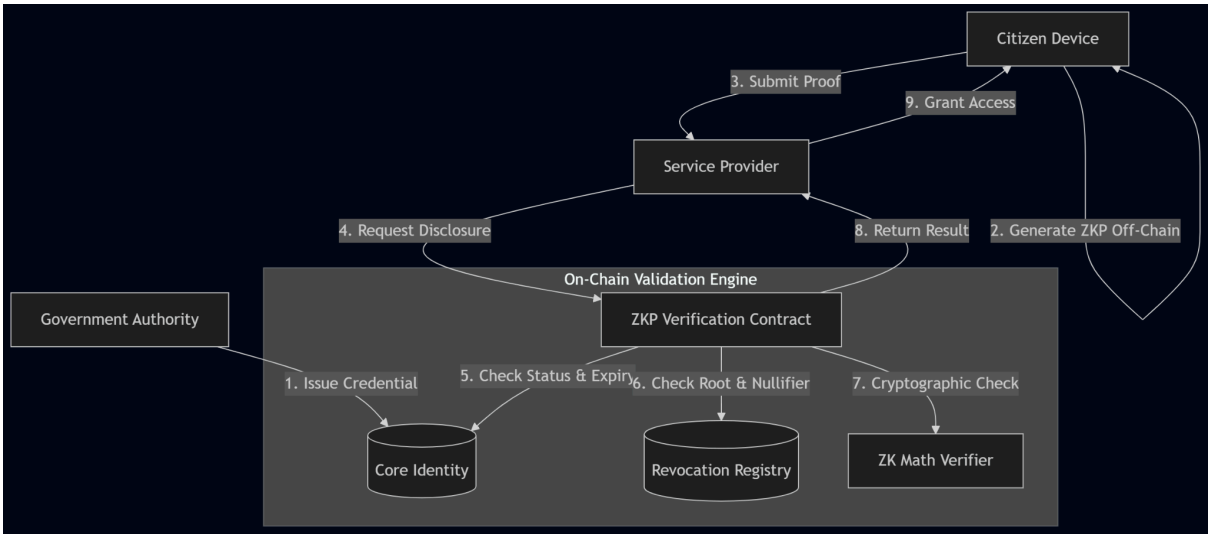
Architectural Overview and Design Choices



The Palestinian National Identity System is built upon a modular smart contract architecture. Rather than relying on a single monolithic contract, the system is divided into three distinct primary contracts, each with strictly defined responsibilities, supplemented by auto-generated mathematical verifiers.



This component diagram provides a high-level structural view of the system's architecture, emphasizing the decoupling between its different layers. It shows the off-chain citizen device acting as the prover, which initiates the interaction by invoking the requestDisclosure function. At the core of the upgradeable proxy layer is the ZKPVerification contract, which operates as the central orchestrator for the entire validation process. The most critical architectural feature highlighted here is how this orchestrator interacts with the rest of the system exclusively through defined interfaces rather than direct contract references. By depending on ICoreIdentityCredential, IRevocationRegistry, and IVerifier, the ZKPVerification contract seamlessly communicates with the CoreIdentityCredential, the RevocationRegistry, and the isolated AgeGroth16Verifier respectively. This modular design guarantees that the identity state, revocation logic, and pure mathematical validation are kept completely separate, allowing any component to be safely upgraded in the future without disrupting the others.



This simplified flow chart outlines the high-level lifecycle of a privacy-preserving identity claim. The process begins with the government authority issuing a credential and anchoring its state within the Core Identity contract. When a citizen needs to prove a claim, such as

their age, they generate a zero-knowledge proof entirely off-chain on their local device, ensuring their raw data never leaves their possession. This proof is then submitted to the service provider, who forwards it to the ZKP Verification contract. At this stage, the on-chain validation engine takes over, executing a strict sequence of checks. It first verifies the credential's active status and expiration date with the core contract, then consults the revocation registry to ensure the credential's nullifier is not blacklisted and that the proof's Merkle root is valid. Only after these state checks are cleared does the system invoke the mathematical verifier to cryptographically validate the proof. Finally, the verification result is returned to the service provider, who can then safely grant the citizen access to the requested service.

At the foundation of the system is the CoreIdentityCredential contract. This serves as the identity anchoring layer, acting as the ultimate source of truth for credential issuance and basic state management. It implements the ICoreIdentityCredential interface to strictly define credential data structures and statuses. Above this sits the ZKPVerification contract, which operates as the privacy and selective disclosure layer. This contract does not hold identity data; instead, it orchestrates the verification process by interfacing with the core identity contract to check active status, the revocation registry for privacy-preserving blacklist checks, and the auto-generated mathematical verifiers (such as the AgeGroth16Verifier) through the IVerifier interface to validate the actual zero-knowledge proofs. Finally, the RevocationRegistry contract acts as a dedicated hybrid manager for privacy-preserving revocations, storing the Merkle root for batch revocations and managing a mapping of nullifiers for immediate emergency revocations.

This strict separation of concerns ensures that the identity database remains isolated from the cryptographic validation logic. If a vulnerability is discovered in the zero-knowledge circuits or if the underlying proving scheme needs to be updated, the ZKPVerification layer can be paused or modified independently without risking the integrity of the national citizen database held within the core contract.

To future-proof this infrastructure, all three primary contracts have been designed using the ERC1967 upgradeable proxy pattern. By utilizing OpenZeppelin's upgradeable libraries and relying on initialization functions rather than standard constructors, the logic of the contracts can be upgraded over time. This architectural choice is critical for a national system: it allows the government to seamlessly deploy a new version of the logic to adapt to new cryptographic standards or business requirements without ever losing or having to migrate the massive storage state of existing citizen credentials.

Regarding the implementation of the selective disclosure and revocation modules, a deliberate design evolution took place to balance efficiency and security. To avoid dangerous inheritance chains and storage collisions commonly associated with upgradeable proxies, the selective disclosure logic, including service provider role management and claim validation, was natively embedded directly within the ZKPVerification contract. Conversely, to maintain logical cohesion and comply with the hybrid architecture requirements, the privacy-preserving revocation mechanisms were decoupled into the standalone RevocationRegistry. This specific distribution of logic reduces unnecessary cross-contract calls where possible while isolating the state management of Merkle roots and nullifiers into a dedicated registry.

To support this modular architecture, the project relies on a technology stack clearly divided between on-chain development and off-chain cryptographic infrastructure. For the smart contract layer, the entire development lifecycle was managed using the Foundry framework. This choice enabled highly efficient writing, compiling, and testing of Solidity contracts, facilitating the creation of rigorous, isolated tests for state management, proxy upgrade simulations, and constant gas optimization monitoring via snapshots. On the privacy front, the zero-knowledge validation logic was entirely designed using the Circom and SnarkJS ecosystem. Circom was employed to mathematically define the attestation circuits, while SnarkJS subsequently handled the entire foundational cryptographic process: from executing the trusted setup using the Groth16 protocol, to generating the mathematical proofs and automatically exporting the Solidity verifier contract. This synergy between frameworks ensured a seamless and secure integration between off-chain processing on the citizen's device and on-chain validation on the blockchain.

Core Identity Credential (Lifecycle Management)

The CoreIdentityCredential contract and its accompanying interface, ICoreIdentityCredential, serve as the foundational layer for managing the lifecycle of digital identities. At the heart of this architecture is the Credential struct. Rather than storing sensitive personal information directly on the blockchain, the system anchors identity using a citizenHash, which acts as a secure zero-knowledge commitment to the citizen's underlying data. This cryptographic approach guarantees that personal data remains entirely off-chain. To heavily optimize gas costs associated with Ethereum's expensive storage operations, the data structures are tightly packed. The struct utilizes uint64 variables for both issuance and expiration timestamps, which, alongside the CredentialType and CredentialStatus enums, are efficiently packed into a single 32-byte storage slot. Furthermore, utilizing enums instead of strings to define the credential category not only reduces gas consumption during data comparisons but also enforces strict type safety at compile time. These anchored identities are centrally managed through a mapping that links a unique identifier to its respective struct.

```
struct Credential {
        bytes32 credentialId;
        bytes32 citizenHash;
        bytes issuerSignature;
        uint64 issuanceTimestamp;
        uint64 expirationTimestamp;
        CredentialType credentialType;
        CredentialStatus status;
    }
```

When an authorized government entity invokes the issueCredential function, the contract validates the parameters, stores the credential on-chain, and immediately broadcasts a CredentialIssued event to notify the network.

```
function issueCredential(
        bytes32 _credentialId,
        bytes32 _citizenHash,
        bytes calldata _issuerSignature,
        uint64 _expirationTimestamp,
        CredentialType _credentialType
    ) external onlyRole(ISSUER_ROLE) whenNotPaused {
```

A particularly interesting architectural trade-off is evident in how the system handles cryptographic validation through the verifyCredentialSignature function. This function reconstructs the signed data hash and utilizes the ECDSA library to recover and verify the issuing authority's address. In standard Solidity development, signature verification is almost universally implemented as a read-only operation utilizing the view modifier, which inherently costs zero gas when called externally. However, in this implementation, the view modifier was intentionally excluded. This decision was driven by a business requirement to maintain an on-chain audit trail of identity usage. By allowing the function to modify the blockchain state, the system can emit the CredentialVerified event every time a signature is checked.

ZKP Verification and Selective Disclosure

The ZKPVerification contract acts as the privacy layer and the primary gateway for service providers. A defining architectural choice in this contract is the abstraction of verification levels. The internal function verifyZKProof is designed as a pure mathematical engine. It remains entirely agnostic to the business logic, simply routing the zero-knowledge proof data (pA, pB, pC, and pubSignals) to the appropriate auto-generated SnarkJS verifier contract via the IVerifier interface.

```
interface IVerifier {
    function verifyProof(
        uint[2] calldata _pA,
        uint[2][2] calldata _pB,
        uint[2] calldata _pC,
        uint[5] calldata _pubSignals
    ) external view returns (bool);
}
```

```
function setVerifier(ICoreIdentityCredential.CredentialType proofType, address verifierAddress) external onlyRole(ADMIN_ROLE) {
        require(verifierAddress != address(0), "Invalid verifier address");
        verifiers[proofType] = IVerifier(verifierAddress);
    }
```

```
function verifyZKProof(
        uint[2] calldata _pA,
        uint[2][2] calldata _pB,
        uint[2] calldata _pC,
        uint[5] calldata _pubSignals,
        ICoreIdentityCredential.CredentialType _proofType
    ) internal view returns (bool) {
        IVerifier verifier = verifiers[_proofType];
        require(address(verifier) != address(0), "Unset verifier for this type");
        return verifier.verifyProof(_pA, _pB, _pC, _pubSignals);
    }
```

Above this sits the verifySelectiveDisclosure function, which acts as the logical bridge. This function extracts the necessary public signals from the proof, enforces specific business rules based on the requested claim type, and strictly binds the zero-knowledge proof to the specific citizenHash stored in the core identity contract. This binding completely mitigates replay attacks, ensuring a malicious user cannot intercept and reuse another citizen's valid proof.

```
function verifySelectiveDisclosure(
    bytes32 _credentialId,
    uint[2] calldata _pA,
    uint[2][2] calldata _pB,
    uint[2] calldata _pC,
    uint[5] calldata _pubSignals,
    ICoreIdentityCredential.CredentialType _claimType
) internal view returns (bool) {
```

To further enhance security, temporal validation is enforced directly within the smart contract. The circuit dictates that the current timestamp is passed as a public signal. The verifySelectiveDisclosure function checks this timestamp against the blockchain's block.timestamp, applying a strict 5-minute (300 seconds) tolerance window. This guarantees that proofs are "fresh" and prevents service providers or interceptors from archiving proofs to gain unauthorized access at a later date.

Access control within this layer adheres strictly to the principle of least privilege. Instead of implementing a generic service provider role, the contract utilizes highly granular roles such as AGE_PROVIDER_ROLE, CITIZENSHIP_PROVIDER_ROLE, and RESIDENCY_PROVIDER_ROLE. This ensures the correct "scope" of requests: a hospital granted the age provider role cannot arbitrarily request citizenship proofs, maintaining strict data compartmentalization. This access control is enforced through the validateDisclosureRequest function, which was deliberately designed to return a boolean value rather than forcing an internal transaction revert. This non-reverting design facilitates pre-transaction queries from decentralized frontends, allowing applications to check authorization status before prompting users to pay gas fees.

```
function validateDisclosureRequest(address _requester, ICoreIdentityCredential.CredentialType _claimType) public view returns (bool) {
    if (_claimType == ICoreIdentityCredential.CredentialType.AgeVerification) {
        return hasRole(AGE_PROVIDER_ROLE, _requester);
    } else if (_claimType == ICoreIdentityCredential.CredentialType.Citizenship) {
        return hasRole(CITIZENSHIP_PROVIDER_ROLE, _requester);
    } else if (_claimType == ICoreIdentityCredential.CredentialType.Residency) {
        return hasRole(RESIDENCY_PROVIDER_ROLE, _requester);
    }
    return false;
}
```

Zero-Knowledge Circuit Design and Proof Generation

The off-chain zero-knowledge logic, which allows citizens to prove claims without revealing data, is implemented in Circom. The age.circom circuit enforces a strict boundary between private knowledge and public validation. The citizen provides their private address, secret salts, birth timestamp, and the Merkle tree path elements. The circuit exposes only three public signals to the blockchain: the current timestamp, the Merkle root, and the immediate revocation nullifier. To validate the age claim, the circuit mathematically ensures the difference between the current timestamp and the birth timestamp is greater than or equal to 18 years, hardcoded as 567648000 seconds.

A critical cryptographic design choice within the circuit is the dual-hash architecture. To ensure perfect compatibility with the Ethereum-native ECDSA signatures managed in the CoreIdentityCredential contract, the circuit utilizes the Keccak256 hash function to compute the citizenHash from the user's address and secrets. However, Keccak is notoriously expensive in zero-knowledge environments, generating a massive number of constraints. To keep proof generation feasible on mobile devices, the circuit switches to Poseidon for calculating the Merkle tree path and the credential nullifier, so the off-chain Merkle Tree must use Poseidon as well. Once the circuit was finalized, the cryptographic verifier was generated using the SnarkJS library. The circuit was compiled and subjected to a Groth16 trusted setup utilizing a Powers of Tau ceremony. This process exported the AgeGroth16Verifier.sol contract, a highly optimized Solidity file containing the verification keys and pairing logic required to validate the proofs on the Ethereum Virtual Machine. For the Foundry testing environment, a fully deterministic proof generation process was established. A valid proof was generated off-chain by locking the current timestamp input to a specific past date (1767268800). Using these mock inputs, the SnarkJS witness was computed, and the resulting cryptographic proof was formatted into Solidity calldata. This produced the exact pA, pB, pC, and pubSignals arrays that are hardcoded into the ZKPVerification.t.sol test suite, allowing the smart contracts to rigorously test the mathematical verifier integration without requiring live proof generation during the continuous integration pipeline.

Within the SnarkJS proving ecosystem, the Groth16 protocol was selected over alternatives like PLONK due to its efficiency in resource-constrained environments. While PLONK offers the architectural advantage of a universal trusted setup, thereby avoiding circuit-specific ceremonies, it inherently produces larger proof sizes and incurs higher verification gas costs on the Ethereum Virtual Machine. Given that this system relies on citizens generating proofs locally on their devices and service providers frequently verifying these transactions on-chain, Groth16's small, constant proof size and minimal gas footprint made it the optimal choice for a highly scalable identity infrastructure.

Hybrid Revocation Registry

This registry acts as the single source of truth for all zero-knowledge-based validations, effectively decoupling privacy-preserving revocations from the administrative tracking found in the core identity contract. A major challenge in verifiable credential systems is managing mass revocations without inflating on-chain storage costs or sacrificing security. This is resolved by computing a "non-revocation" Merkle tree entirely off-chain, where the leaves

represent active, valid credentials. Only the 32-byte Merkle root is recorded on the blockchain. Crucially, the validation of the Merkle path is executed entirely inside the zero-knowledge circuit. This design choice provides absolute cryptographic certainty that a credential belongs to the unrevoked set, without burdening the Ethereum Virtual Machine with expensive proof verifications.

```
bytes32 public merkleRoot;

function updateMerkleRoot(bytes32 _newRoot) external onlyRole(REVOKER_ROLE) whenNotPaused {
```

While Merkle trees are highly efficient for batch operations, they inherently lack the speed required for emergency interventions. To address this, the system implements an immediate revocation mechanism using nullifiers. A nullifier is a deterministic, one-way hash of the credential's leaf. If a device is stolen or a credential compromised, the issuing authority can invoke the revokeImmediately function to instantly blacklist the specific nullifier. Because the nullifier cannot be reverse-engineered to reveal the citizen's original identity data, this acts as a targeted emergency kill switch that perfectly preserves the zero-knowledge property and prevents linkability across different service providers.

```
mapping(bytes32 => bool) public immediateRevocations;
```

Behind the scenes, the off-chain zero-knowledge circuit utilizes a dual-hash architecture to make mobile proving feasible. The EVM-native Keccak256 algorithm is employed exclusively to generate the citizenHash. This guarantees perfect compatibility with the ECDSA cryptographic signatures verified on-chain by the core identity contract. Conversely, the heavily constrained Merkle tree traversals and nullifier generation utilize the Poseidon hash function. Poseidon is specifically designed for zero-knowledge environments and drastically reduces the number of constraints required by the circuit, ensuring that generating a proof remains computationally light and fast for citizen devices.

Finally, these revocation constraints are enforced on-chain through a strict "Fail-Fast" validation pattern designed to maximize gas efficiency. Within the ZKPVerification contract, before the computationally expensive mathematical proof is evaluated, the system invokes the internal _checkCredentialIsActive function. This function extracts the merkleRoot and the nullifier directly from the proof's public signals. It then queries the registry to verify that the nullifier is not blacklisted and that the provided root precisely matches the latest on-chain state. If a credential is revoked or outdated, the transaction immediately reverts. This fail-fast mechanism saves the service provider the massive gas fees associated with processing an invalid Groth16 cryptographic pairing.

Emergency Management (Pausable Pattern)

To safeguard the national infrastructure against catastrophic scenarios, the system incorporates the Pausable pattern. Crucially, this security measure is compartmentalized. It is implemented independently across the core identity and verification contracts rather than relying on a cascading global lock. This deliberate design avoids a domino effect: if the

zero-knowledge verification layer is compromised, the government can pause it while continuing to securely register new identities in the core contract. Conversely, if the government's issuing infrastructure is breached, the verification layer remains operational, allowing hospitals and service providers to validate previously issued credentials without paralyzing the entire country.

In an issuance compromise scenario, such as the theft of government administrative keys, the core identity contract can be instantly paused to strictly block any new credential issuance or administrative revocations. However, a fundamental architectural choice was made to intentionally exclude pause restrictions from read-only and cryptographic validation functions, most notably the signature verification process. This ensures that while the attack is contained at the issuance level, citizens holding valid, untampered credentials can still access essential primary services uninterrupted.

Testing Strategy and Code Coverage

To guarantee the highest level of security and reliability for the national identity infrastructure, a comprehensive testing strategy was implemented utilizing the Foundry framework. The testing suite was designed not only to verify the "happy paths" of the system but to rigorously stress-test edge cases, cryptographic validations, and access control boundaries.

A significant architectural decision during the testing phase was the implementation of the Test Harness pattern. In smart contract development, it is critical to test isolated internal logic, but exposing internal functions as public in the main contract increases the attack surface and inflates deployment gas costs. To solve this, a dedicated ZKPVerificationHarness contract was created that inherited from the main verification contract. This harness explicitly exposed critical internal functions, such as the fail-fast credential checks, the pure mathematical proof verification, and the selective disclosure logic. This allowed the testing suite to rigorously validate these isolated components without compromising the security or efficiency of the production code.

```
contract ZKPVerificationHarness is ZKPVerification {
    function exposeCheckCredentialIsActive(bytes32 credentialId, uint[5] calldata _pubSignals) external view {
        _checkCredentialIsActive(credentialId, _pubSignals);
    }
```

Furthermore, given the system's reliance on the ERC1967 proxy pattern, advanced upgradeability testing was mandatory. To ensure that future logic upgrades would not corrupt existing citizen data or cause storage collisions, specific upgradeability tests were written for both the Core and ZKP contracts. By directly manipulating the Ethereum Virtual Machine state, the tests manually overwrite the exact EIP-1967 implementation storage slot with the address of a mock "V2" contract. The tests then successfully invoke new functions on the upgraded logic, proving that the proxy routing remains intact and state transitions are safe.

```
// Upgradeability mock contract
contract ZKPVerificationV2 is ZKPVerificationHarness {
    function getVersion() external pure returns (string memory) {
        return "V2";
    }
}
```

The testing suite exceeds the required 90% coverage threshold by targeting branch coverage and failure conditions. Extensive negative testing was implemented to verify that unauthorized access attempts revert. This includes testing that non-issuers cannot forge credentials, unauthorized entities cannot access selective disclosures, and service providers cannot request scopes outside their jurisdiction. Additionally, catastrophic emergency scenarios were simulated to confirm that when the contracts are paused, all state-modifying functions and disclosure requests are properly blocked by the pause mechanisms.

To achieve this coverage efficiently, several testing optimizations were employed. Temporal manipulation was streamlined by utilizing Foundry's deep state isolation. Rather than redundantly manipulating the blockchain time in every individual test, the system's clock was warped to a specific past timestamp directly within the setup function. Because Foundry cleanly resets the state before each test block, this ensured a perfectly synchronized and highly optimized environment for validating the hardcoded zero-knowledge proof public signals. Finally, the core identity tests leveraged Foundry's native cryptographic cheat codes to generate dynamic ECDSA signatures, rigorously validating the core contract's ability to identify both tampered data and invalid signers.

File	% Lines	% Statements	% Branches	% Funcs
src/AgeVerifier.sol	91.04% (61/67)	90.48% (57/63)	0.00% (0/3)	100.00% (4/4)
src/CoreIdentityCredential.sol	100.00% (33/33)	100.00% (32/32)	100.00% (13/13)	100.00% (7/7)
src/RevocationRegistry.sol	100.00% (22/22)	100.00% (15/15)	100.00% (6/6)	100.00% (7/7)
src/ZKPVerification.sol	85.71% (54/63)	86.67% (52/60)	70.27% (26/37)	84.62% (11/13)
test/CoreIdentityCredential.t.sol	100.00% (2/2)	100.00% (1/1)	100.00% (0/0)	100.00% (1/1)
test/ZKPVerification.t.sol	100.00% (8/8)	100.00% (6/6)	100.00% (0/0)	100.00% (4/4)
Total	92.31% (180/195)	92.09% (163/177)	76.27% (45/59)	94.44% (34/36)

Regarding the detailed metrics, it is worth noting that the branch coverage percentage for the ZKPVerification contract is naturally lower compared to the rest of the test suite. This discrepancy does not indicate a lack of testing, but is a direct consequence of the scope of this initial implementation. The logical branches dedicated to verifying citizenship and residency claims were not executed during the tests because the system is currently focused strictly on validating the zero-knowledge circuit for age verification. Consequently, these blocks were temporarily isolated with intentional reverts, pending future cryptographic integrations.

Gas Optimization Tests

Given the economic constraints of deploying and operating smart contracts on the Ethereum Virtual Machine, rigorous gas optimization was a primary focus throughout the development lifecycle. The primary mechanism for monitoring these metrics was Foundry's snapshot feature. By routinely executing the forge snapshot command, the framework generated a comprehensive baseline of the gas consumed by every individual test. This baseline acted as an automated regression guardrail. Whenever new logic was introduced, such as the fail-fast validation pattern or the hybrid revocation checks, comparing the new snapshot against the previous baseline immediately highlighted any unintended increases in computational cost.

Furthermore, to maintain continuous visibility into the operational costs of the architecture, the testing environment was configured to generate automated terminal reports. By modifying the Foundry configuration file to enable comprehensive gas tracking, every execution of the test suite automatically output a detailed breakdown of execution costs. This report provided the minimum, average, and maximum gas used for all state-modifying functions, alongside the exact deployment costs for the upgradeable proxies and their underlying implementations.

src/CoreIdentityCredential.sol:CoreIdentityCredential Contract					
Deployment Cost	Deployment Size				
2098528	9783				
Function Name	Min	Avg	Median	Max	# Calls
ISSUER_ROLE	370	370	370	370	26
credentials	3489	10329	11489	17867	18
grantRole	30261	30261	30261	30261	45
initialize	50507	50507	50507	50507	45
issueCredential	4017	98455	98698	165290	93
pause	3045	18687	26508	26508	3
paused	2635	2635	2635	2635	2
revokeCredential	5367	14131	14641	17448	33
unpause	3067	6246	6246	9426	2
verifyCredentialSignature	10656	20867	25910	25910	5

Deployment and Configuration Process

Transitioning the privacy-preserving identity system from a local testing environment to a live blockchain requires a deterministic and highly reproducible orchestration strategy. To achieve this, the project utilizes Foundry's native Solidity scripting capabilities. Rather than

deploying contracts manually, which is inherently prone to human error, a dedicated deployment script dictates the exact chronological sequence required by the upgradeable architecture. For each of the three primary modules, the script first deploys the underlying logic implementation. Immediately afterward, it deploys the corresponding ERC1967 proxy and atomically invokes the initialization function. This strict sequence is critical to successfully secure the initial state and eliminate the risk of front-running initialization attacks.

Once the core infrastructure is established, the script handles the complex post-deployment wiring. The ZKP Verification orchestrator is explicitly injected with the on-chain addresses of the newly deployed Core Identity and Revocation Registry contracts, establishing the interfaces required for the validation flow. Subsequently, the autogenerated mathematical AgeGroth16Verifier is deployed and formally linked to the verification layer using the designated setter functions. To finalize the system's readiness, the script outlines the assignment of essential cryptographic roles via the AccessControl module, such as granting the designated government authority the ability to issue credentials and manage revocations.

It is important to note that while this deployment script is fully developed and included in the final source code to demonstrate the correct architectural choreography, it is presented as a theoretical blueprint. A corresponding environment file containing active private keys or network RPC endpoints was intentionally omitted, as live deployment accounts or mock funded wallets were not provisioned for this specific development phase.

To practically execute this deployment orchestration, a network operator would first populate the environment variables, and then utilize Foundry's `forge script` command. The execution on a live network (for example, the Sepolia testnet) would follow this syntax:

```
forge script script/Deploy.s.sol:DeploySystem --rpc-url $SEPOLIA_RPC_URL
--broadcast --verify
```